

Multi-UAV air combat cooperative game based on virtual opponent and value attention decomposition policy gradient[☆]

Xiaojie Xu^{a,b}, Yunfan Wang^{a,b}, Xian Guo^{a,b,*}, Kuihua Huang^c, Xuebo Zhang^{a,b}

^a Institute of Robotics and Automatic Information System, College of Artificial Intelligence, Nankai University, Tianjin, China

^b Tianjin Key Laboratory of Intelligent Robotics, Nankai University, China

^c Laboratory for Big Data and Decision, College of Systems Engineering, National University of Defense Technology, China

ARTICLE INFO

Keywords:

Air combat cooperative game
Reinforcement learning
Value attention decomposition

ABSTRACT

In the multi-unmanned aerial vehicle (UAV) air combat confrontation environment, deriving the cooperative policy of friendly aircraft is still a challenge, owing to the higher-order differential dynamics model of aircraft and the confidence assignment problem in multi-UAV air combat with conflict and cooperation. In this paper, a novel reinforcement learning method that combines virtual opponent and value attention decomposition is proposed. In particular, to reduce the difficulty in training induced by the higher order differential dynamics model, the actions of aircraft are abstracted into actions of the game layer and maneuvering actions of the bottom layer, in which the actions of the game layer are modeled as the pose of the virtual opponent. In the training process, only the policy of the game layer is trained, and the maneuvering policy of the bottom layer is the default policy or the rule-based policy. To address the confidence assignment problem encountered during multi-UAV cooperative training, the total value function of the team is decomposed into individual value functions based on the attention mechanism, and the policy of the game layer is optimized by integrating the individual value into the gradient computation as the baseline. Finally, the algorithm is verified on the dynamic high-fidelity training platform. The results indicate that the algorithm outperforms the state-of-the-art method in typical multi-UAV air combat scenarios such as 4V4, 5V5, and 6V6.

1. Introduction

With the development of deep learning and deep reinforcement learning, artificial intelligence (AI) has made a profound impact on the unmanned aerial vehicle (UAV) system, promoting the application of UAVs in various fields. For example, in the field of autonomous navigation, AI enables the UAV to learn navigation through an unknown environment (Pham, La, Feil-Seifer, & Nguyen, 2018). Moreover, autonomous control of the UAV can be realized through deep reinforcement learning (Carrio, Sampredo, Rodriguez-Ramos, Campoy, et al., 2017). However, collaboration in a multi-UAV air combat game is still a challenge owing to the complex aerodynamics and high-dimensional joint continuous action space.

In recent years, deep reinforcement learning has realized significant achievements in a variety of games, such as Alpha Go (Silver et al., 2017), OpenAI-Five (Berner et al., 2019), AlphaStar for StarCraft (Oriol et al., 2019), Suphx for Mahjong (Junjie et al., 2020), and DeepNash for Stratego (Julien et al., 2022). Multi-UAV air combat is a scenario where military aircraft fight against each other over a wide airspace. It is becoming an increasingly important combat environment in multi-agent

game (Yang, Yang, & Yu, 2024). In contrast to the abovementioned games, the difficulties encountered in the multi-UAV air combat game are as follows:

- Higher order coupling policy of maneuvering and game: The maneuverability and velocity of a military aircraft are very high, and its movement is driven by higher order differential dynamics, which involves complex and uncertain aerodynamics. The state is very sensitive to the maneuvering action; therefore, maintaining steady flight is very hard. However, in addition to steady flight, an aircraft in combat must fire at the enemy aircraft and simultaneously avoid being shot down. Therefore, directly learning the maneuvering policy in air combat is generally very difficult.
- Confidence assignment with conflict and cooperation: Multi-UAV air combat is a confrontational game. The UAV must simultaneously cooperate with friendly aircraft to maximize team revenue and fight against the enemy, which further increases the difficulty of confidence assignment and learning.

[☆] This work is supported by the National Natural Science Foundation of China (62073176).

* Corresponding author at: Institute of Robotics and Automatic Information System, College of Artificial Intelligence, Nankai University, Tianjin, China.

E-mail addresses: 2120220490@mail.nankai.edu.cn (X. Xu), 2120210376@mail.nankai.edu.cn (Y. Wang), guoxian@nankai.edu.cn (X. Guo), khhuang@nudt.edu.cn (K. Huang), zhangxuebo@nankai.edu.cn (X. Zhang).

<https://doi.org/10.1016/j.eswa.2024.126069>

Received 25 May 2024; Received in revised form 27 November 2024; Accepted 3 December 2024

Available online 19 December 2024

0957-4174/© 2024 Elsevier Ltd. All rights reserved, including those for text and data mining, AI training, and similar technologies.

As described above, one of the challenges of multi-UAV air combat is the higher order coupling policy of maneuvering and game, which makes learning directly by mapping from observations to maneuvering actions very difficult. Therefore, hierarchical reinforcement learning methods are proposed for air combat. These generally consist of a top-level strategy selector and a bottom-level maneuver policy (Adrian et al., 2023; Chenxu, Xuebo, Lun, Minghui, & Yongchun, 2024; Haiyin et al., 2023). In general, the top-level policy is used to choose the switching strategy between attack and defense (Chenxu et al., 2024) or discrete maneuver from a finite set (Adrian et al., 2023), and the bottom-level policy controls the maneuver flight.

The hierarchical design of the action space can avoid the need to learn the higher order coupling policy directly, which reduces the level of training difficulty. However, the top-level policy is generally discrete, which limits the flexibility and optimality. In this study, the action space is also hierarchical, that is, it contains top-level game actions and bottom-level maneuver actions. In contrast to the references listed above, the top-level actions are not discrete but the continuous six-degree-of-freedom pose of the virtual opponent overcomes the limits.

Another challenge of multi-UAV air combat is the confidence assignment with conflict and cooperation. To address the confidence assignment of multiple agents, several multi-agent reinforcement learning methods have been developed recently. The most well-known algorithms are value decomposition algorithms such as VDN (Peter et al., 2017), QMIX (Tabish et al., 2020), and QTRAN (Kyunghwan, Daewoo, Wan, David, & Yung, 2019). However, these methods are only suitable for low-speed agents with discrete actions, which cannot be used for the UAV. There are also multi-agent reinforcement learning algorithms that can deal with continuous actions such as multi-agent proximal policy optimization (MAPPO) (Chao et al., 2022) and FACMAC (Bei et al., 2021), but they are not suitable for the UAV because of its higher order dynamics.

For multi-UAV air combat, CoPPO is used to achieve cooperation of multiple UAVs by projecting the objective and clipping the overall optimization step (Haiyin et al., 2023). Unlike other fully cooperative multi-agent systems, the UAV needs to avoid being killed. Therefore, the contributions to the total value of the team are complex and significant. In this study, attention mechanism is adopted to decompose the total value function into the individual value function, which is then introduced to the policy gradient.

In the latest study, builds upon the foundation of MAPPO, MP3O is proposed as an efficient MARL algorithm (Zheng, Xin, He, & Ding, 2024), designed to enhance cooperative policy learning in addressing the multi-UAV air combat problem.

Different from previous works, we aim to propose a hierarchical policy to improve learning efficiency and solve the confidence assignment problem. The main contributions of our work are summarized below:

- A novel virtual opponent based hierarchical policy is proposed; its top-level action is the continuous six-degree-of-freedom pose of the enemy, overcoming the discrete limits and efficiently exploring in a continuous state space.
- A value attention decomposition policy gradient algorithm is developed to optimize the policy networks that can solve the credit assignment problem of multi-UAV air combat with conflict and cooperation.
- Experiments are performed on the dynamic high-fidelity training platform. The results indicate that the proposed method outperforms the state-of-the-art multi-agent reinforcement learning in typical air combat such as 3v3, 4V4, 5V5, and 6V6.

The rest of this paper is organized as follows. In Section 2, we begin with a brief introduction to the work carried out so far on air combat solutions. In Section 3, cooperative multi-agent reinforcement learning is discussed. We describe the proposed approach in detail in Section 4. The details of the experimental section are described in Section 5. Finally, a conclusion is drawn in Section 6.

2. Related work

In this section, related work on the multi-UAV air combat cooperative game will be introduced in detail, including the AI-based method on air combat, hierarchical reinforcement learning, and multi-agent collaboration.

2.1. AI-based method on air combat

Since the 1950s, extensive research has been dedicated to developing algorithms capable of autonomously conducting air combat (Isaacs, 1951). In the study of rule-based methods, Burgin, Fogel, and Phelps (1975) developed adaptive maneuvering logic for one-on-one air combat environments. Following the rule-based work, several mathematical modeling based methods have been proposed. Harmon, Baird, and Klopff (1995) presented an application of reinforcement learning to a linear-quadratic differential game, utilizing the residual-gradient form of advantage updating algorithm. Virtanen, Karelaiti, and Raivio (2006) proposed a multistage influence diagram game method for modeling the maneuvering decisions of pilots in one-on-one air combat. The tactical battle manager (Floyd, Karneeb, Moore, & Aha, 2017) was utilized to control an autonomous UAV in simulated beyond-visual-range (BVR) air combat. Further, numerous methods based on intelligent learning have been proposed in recent years. Ernest et al. (2016) introduced ALPHA, an AI system controlling flights of UAVs in aerial combat missions, which defeated Colonel (retired) Gene Lee in 2016. Pope et al. (2022) used a novel hierarchical deep reinforcement learning approach.

2.2. Hierarchical reinforcement learning

Owing to the dimensional curse that the number of parameters to be learned grows exponentially with the size of any compact encoding of a state, hierarchical methods of reinforcement learning have been proposed (Barto & Mahadevan, 2003). Inspired by ideas from machine learning, a new perspective on hierarchical behavior has begun to take shape, giving rise to some thought-provoking new findings (Botvinick, 2012). In recent research, a three-level hierarchical framework embedding expert knowledge was raised to solve the complex dynamic constraints and long decision-making process problem.

2.3. Multi-agent collaboration

In recent years, new methods have been proposed for multi-agent cooperative air combat training. Jiandong, Qiming, Guoqing, Yi, and Yong (2021) used a bidirectional recurrent neural network and combined it with target allocation and air combat situation assessment to generate a cooperative tactical maneuver policy. Liu, Sun, Tao, Shou, and Xu (2021) established the assessment model to evaluate the situation of each drone in the combat scenario, and the modified estimate of distribution algorithm (EDA) was proposed to solve the target allocation problem. Thereafter, an air combat cooperative strategy framework (Gong, Xu, & Luo, 2023) was proposed based on the networked decentralized partially observable Markov decision process.

As described above, EDA and other methods have been introduced for multi-agent cooperative air combat training, but the current research still faces the training difficulties introduced by the high-order differential dynamics model and the challenge of confidence allocation among team agents. Therefore, we aim to derive a cooperative policy applicable to friendly aircraft that can address the training challenges.

3. Preliminaries

3.1. Fully cooperative multi-agent task

A fully cooperative multi-agent task can be described as a tuple $\langle S, U, P, R, Z, O, n, \gamma \rangle$ defined in a random game G , in which n agents can be represented by the superscript selected as the label; the superscript $-a$ is defined as all other agents that do not contain A , and the state in the multi-agent environment is $s \in S$. At each decision time instant t , each agent chooses an action to form a joint action with the goal of maximizing the team's cumulative reward. This joint action acts on the state transition equation of the environment, resulting in a transition from the current state to the next state: $P(s_{t+1}|s_t, \mathbf{u}_t) : S \times U \times S \rightarrow [0, 1]$. Since it is a fully cooperative environment, all agents use the same reward function to receive the same reward $R(s_t, \mathbf{u}_t, s_{t+1}) : S \times U \rightarrow \mathcal{R}$ as the team at time instant t . The objective of the agent is to maximize the expected cumulative return of the discount $\mathcal{R}_t := \sum_{i=1}^{\infty} \gamma^{i-1} R_t$, where γ is the discount factor.

In the fully observable environment, each agent calculates the action through its own strategy π_a and the global state s , in which case there is an optimal strategy, whereas in the partially observable environment, the agent obtains the local observation based on the observation function. Every agent has an action-observed historical trajectory, and the optimal strategy in partially observable environments may depend on the historical observations and actions of the agent.

3.2. Reinforcement learning

Reinforcement learning (RL) is a branch of machine learning that focuses on training agents to make sequential decisions to achieve a cumulative reward. RL can be mathematically formulated as a Markov decision process, which consists of a tuple (S, A, P, R, γ) , where S is the set of states in the environment; A is the set of actions the agent can take; P is the state transition probability function $P(s'|s, a)$, which gives the probability of transitioning to state s' when taking action a in state s ; R is the reward function $R(s, a, s')$, which represents the immediate reward received after transitioning from state s to state s' by taking action a ; and γ is the discount factor, which balances the immediate and future rewards. The goal of the agent is to learn a policy π , which is a mapping from states to actions, that maximizes the expected cumulative reward. The value function $V^\pi(s)$ represents the expected cumulative reward when starting in state s and following policy π thereafter:

$$V^\pi(s) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t R_{t+1} \mid s_0 = s \right] \quad (1)$$

where R_{t+1} is the reward received at time step $t+1$. The optimal value function $V^*(s)$ represents the maximum expected cumulative reward achievable from state s onwards:

$$V^*(s) = \max_{\pi} V^\pi(s) \quad (2)$$

The optimal action-value function $Q^*(s, a)$ represents the maximum expected cumulative reward achievable from state s by taking action a and following the optimal policy thereafter:

$$Q^*(s, a) = \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t R_{t+1} \mid s_0 = s, a_0 = a \right] \quad (3)$$

The Bellman optimality equation is used to relate the optimal value function to itself recursively:

$$Q^*(s, a) = \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma \max_{a'} Q^*(s', a')] \quad (4)$$

Solving this equation enables the agent to determine the optimal policy π^* , which achieves the maximum expected cumulative reward.

3.3. BVR multi-UAV air combat environment

In this section, we describe the building of a multi-UAV air combat confrontation environment that supports the total number of red and blue UAVs below 12. The high-fidelity UAVs multi-UAV air combat confrontation environment is used. It is developed based on the open source aircraft dynamics model software JSBSim, which supports the acquisition of the aircraft's position (longitude, dimension, and altitude), attitude (yaw, pitch, and roll angles), speed (three axial speeds), angle of attack, and sideshift angle. The control actions of each UAV in the environment are aileron, rudder, elevator, and throttle. The agent strategy controls the interaction between the UAV and the environment to obtain local observations and rewards.

In this multi-UAV air combat adversarial environment, the decision problem is usually locally observable and the reward signal is shared globally, which belongs to a distributed partially observable Markov decision process. If the single-agent algorithm is directly applied to the multi-agent environment, then for any one agent, all other agents are regarded as part of the environment; each agent is constantly updating the strategy, and the state transfer and reward of the environment depend on the strategies of all the agents. Therefore, each agent will constantly update its own strategy to adapt to the environment composed of all other agents, resulting in the nonstationarity of the environment. To avoid the nonstationarity of the environment, we use the mainstream algorithm framework of centralized training distributed execution to design the algorithm. The global state of the environment must be used in the training process, and each agent must make decisions based only on the local observations obtained in the execution stage. With this context in mind, we set the local observation of agent a as:

$$\begin{aligned} o^a &= \{o_{\text{self}}, o_{\text{friends}}, o_{\text{opponents}}\} \\ o_{\text{friends}} &= \{o_{\text{friend}_1}, o_{\text{friend}_2}, \dots, o_{\text{friend}_{n_f}}\} \\ o_{\text{opponents}} &= \{o_{\text{opponent}_1}, o_{\text{opponent}_2}, \dots, o_{\text{opponent}_{n_o}}\} \end{aligned} \quad (5)$$

Here, o^a represents the local observation space and o_{self} represents some states of the current UAV itself. Specific information is presented in the following equation:

$$o_{\text{self}} = \left\{ \underbrace{x, y, z}_{\text{position}}, \underbrace{\phi, \theta, \psi}_{\text{posture}}, \underbrace{h}_{\text{health}} \right\} \quad (6)$$

where o_{friends} is the status information of all the teammates, n_f is the number of teammates, and the status information of each teammate includes position, posture, and health; $o_{\text{opponents}}$ is the status information of all the opponents, n_o is the number of opponents, and the status information of each opponent includes position, posture, and health. Each UAV can only observe its teammates and opponents in its neighborhood through radar information. If it cannot observe teammates or opponents, the corresponding position is set to 0. If a teammate or opponent is dead, the corresponding position is 0. In the status information of each UAV, x, y, z are the position information in three dimensions, ϕ, θ, ψ are the attitude information in three dimensions, and h is the health of the UAV. The global state is the concatenation of the local observations of all agents, and the global state space is the multiplicative of the local observation space of each agent: $S = \prod_{i=1}^n O_i$. The global state is represented by Eq. (3). When using local observations and the global state for training, the maximum and minimum values must be normalized:

$$s = \{o^1, o^2, \dots, o^a, \dots, o^n\} \quad (7)$$

The actions of agent a in the environment are as follows:

$$u^a = \{\delta_a, \delta_r, \delta_e, \delta_t\} \quad (8)$$

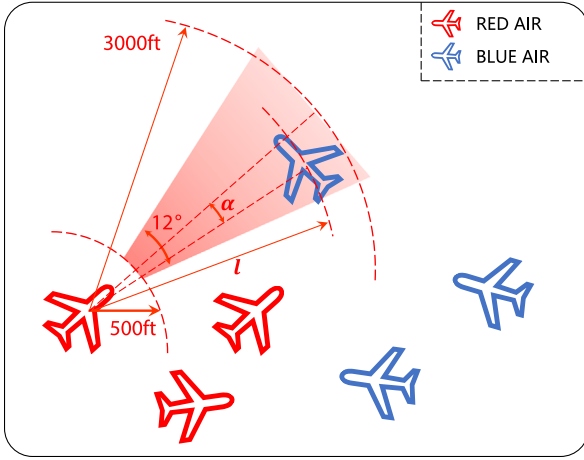


Fig. 1. Weapon attack area.

where δ_a , δ_r , δ_e , and δ_t represent the ailerons, rudder, elevator, and throttle, respectively. The input of the control action is continuous, and the maximum and minimum values of the action are as follows:

$$\begin{aligned} u_{\max}^a &= [1, 1, 1, 1] \\ u_{\min}^a &= [-1, -1, -1, 0] \end{aligned} \quad (9)$$

In the multi-UAV air combat environment, the red and blue UAVs engage in game confrontation. Before the confrontation starts, a health value of 100 is initialized for each UAV. During the confrontation, the damage caused is calculated according to the position of the drone in the attack area of the opponent's weapon. As illustrated in Fig. 1, the weapon attack area is defined as the position of all the points within the spherical cone with a 12-degree aperture. If the blue drone is within the weapon attack area of a red UAV, the blue UAV will be attacked by the red UAV. The specific damage value is calculated as follows:

$$\begin{aligned} d_{ri,bj}^{\text{total}} &= d_{ri,bj}^{\text{distance}} * d_{ri,bj}^{\text{angle}} \\ d_{ri,bj}^{\text{distance}} &= \begin{cases} 0 & l > 3000\text{ft} \\ \frac{3000-l}{2500} & 500\text{ft} \leq l \leq 3000\text{ft} \\ 0 & l < 500\text{ft} \end{cases} \\ d_{ri,bj}^{\text{angle}} &= \begin{cases} 0 & \|\alpha\| > 6^\circ \\ \frac{6-\|\alpha\|}{6} & 0^\circ \leq \|\alpha\| \leq 6^\circ \end{cases} \end{aligned} \quad (10)$$

where $d_{ri,bj}^{\text{total}}$ represents the damage caused by the i th UAV of the red side to the j th UAV of the blue side. The damage should be calculated as the product of the distance damage $d_{ri,bj}^{\text{distance}}$ and the angle damage $d_{ri,bj}^{\text{angle}}$. l is the distance between the attacking UAV and the attacked UAV, and α is the deflection angle of the center line of the blue attacked UAV and the red attacking UAV.

In the environment of our study, the blue UAVs are regarded as the environment, and all the red agents need to cooperate and fight against the blue UAVs to maximize the average cumulative reward of the red team. During the confrontation, all the red agents will receive the same reward for each decision made by the red team. The reward received by the red agent in step t can be calculated using the following formula:

$$r_t = d_{r,b,t} - d_{b,r,t} \quad (11)$$

The greater the damage caused by the red side to the blue side or the smaller the damage caused by the blue side to the red side, the higher the reward received by the red side. $d_{b,r,t}$ is the sum of the damage caused by all the blue side drones to all the red side drones in step t ,

and $d_{r,b,t}$ is the sum of the damage caused by the red side to the blue side:

$$\begin{aligned} d_{b,r,t} &= \sum_{i=1}^{n_b} \sum_{j=1}^{n_r} d_{bi,rj}^{\text{total}} \\ d_{r,b,t} &= \sum_{i=1}^{n_r} \sum_{j=1}^{n_b} d_{ri,bj}^{\text{total}} \end{aligned} \quad (12)$$

4. Method

In multi-UAV air combat, the main difficulty is not that the algorithm cannot efficiently explore the desired target state owing to the long-field task, but that the complexity of the high-fidelity flight dynamics model of UAV and the continuity of the state action space lead to difficulty in algorithm exploration. Direct training will lead to a large number of ineffective explorations. The initial random strategy will lead to UAV roll, stall, and crash, and the data generated by the interaction between the agent and the environment are all negative reward data. Learning flight, confrontation, and multi-UAV air combat confrontation in such an environment, where the state action space is continuous and the flight dynamics model is constrained, is very difficult.

In this section, we adopt a hierarchical idea to reduce the difficulty in multi-UAV air combat confrontation training, including two key components: action modeling method by virtual opponent and value attention decomposition.

4.1. Action modeling method by virtual opponent

Multi-UAV air combat confrontation strategies are divided into upper- and lower-level strategies. At each decision-making moment, the upper-level strategies output upper-level action instructions, and the lower-level strategies generate real actions after receiving the upper-level action instructions. This hierarchical method is different from the hierarchical method of time series abstraction because the latter combines low-level decisions of multiple time steps with an unfixed number into upper-level decision, whereas the hierarchical method described here implies that both the upper- and lower-level strategies must make decisions at each decision-making moment. This method can also be understood as a special case of time series abstraction. Another difference is that the continuous state action space leads to difficulty of exploration. The lower-level strategies described here use pretrained strategies, and only the upper-level strategies have to be trained, whereas the hierarchical reinforcement learning of temporal abstraction requires the training of strategies at various levels.

To simplify UAV multi-UAV air combat confrontation training, we use the hierarchical idea to decompose the confrontation into one-to-one confrontations and the pretraining strategy to simplify the exploration. Fig. 2 depicts the proposed framework of multi-UAV air combat confrontation based on hierarchical thinking.

The first layer of the top layer is the multi-UAV air combat confrontation environment of red and blue parties. In the figure, there are three red drones on the left and three blue drones on the right. The second layer is the multi-agent strategy of the upper layer. In the multi-agent environment, each agent strategy is constantly changing, which leads to the nonstationarity of the environment. Therefore, the multi-agent algorithm of this layer uses the training paradigm of centralized training and distributed execution. The multi-agent strategy receives the observation of the upper-layer environment and outputs the upper-layer action that each agent needs to perform following the multi-agent strategy. In the third layer, the actions output by the multi-agent strategy generate a virtual opponent drone for each red drone, with three red drones on the left and three real blue drones on the right. The smaller dotted light blue drone generates a virtual opponent drone for the multi-agent strategy, and each red drone has a virtual opponent.

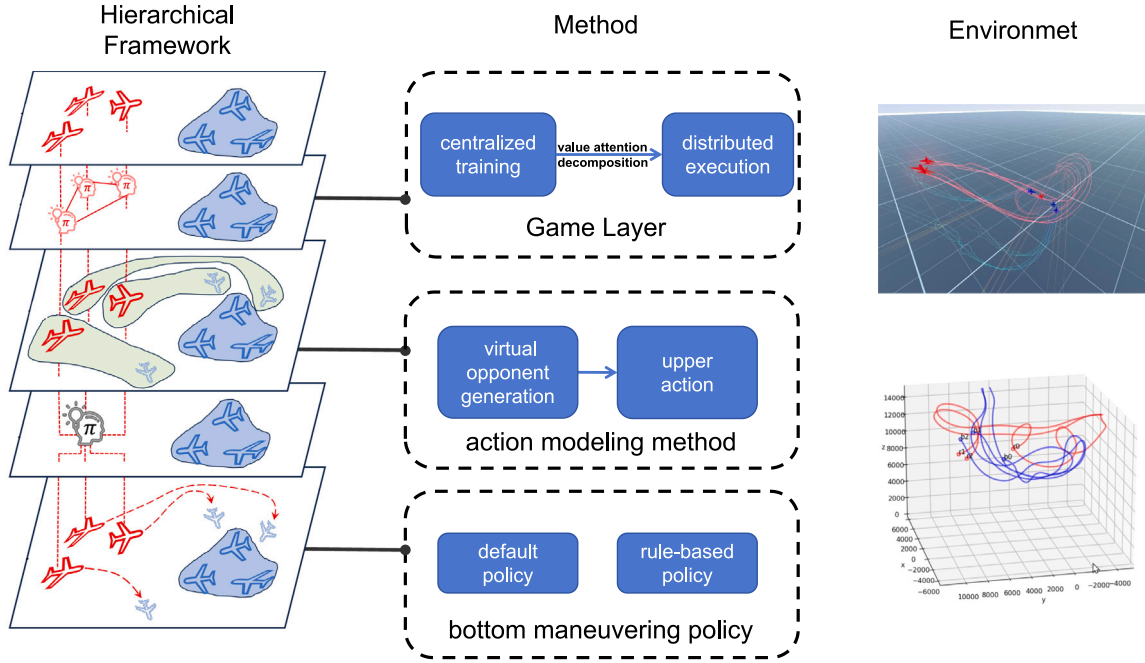


Fig. 2. Hierarchical strategy framework: In the hierarchical framework, the top layer is the real confrontation environment, and the second layer is the game layer, which allocates distributed execution by means of centralized training and value attention decomposition. The third layer is the action modeling method, which generates the upper action space through the virtual opponent. The fourth and fifth layers are the underlying control strategies and virtual one-to-one confrontation based on the default or rule-based strategy.

The third layer is indicated by a green background. In the third layer, the multi-agent confrontation is decomposed into multiple sets of one-to-one confrontation problems. In the fourth layer, the red UAV state of each group of one-to-one confrontation and the corresponding virtual blue UAV state are taken as inputs, and the pretrained one-to-one confrontation strategy is used to output the commands controlling the UAV flight (ailerons, rudder, elevator, and throttle). The fifth layer generates the control instructions of the ascension on the environment, carries out state transfer, completes the one-to-one confrontation between multiple groups of red drones and virtual blue drones, and indirectly realizes the task of red multi-agent cooperation against blue drones.

The upper-layer multi-agent strategy addresses the nonstationarity problem of the multi-agent environment through centralized training and distributed execution. By observing the multi-UAV air combat confrontation situation at the macro level, a cooperative strategy is formulated to plan a virtual opponent for each red UAV. The upper layer policy generates virtual opponents based on the scenario rather than the number of opponents, so it can generate as many virtual opponents as the number of red agents in every scenario. The lower-layer strategy uses the pretrained one-to-one confrontation strategy to achieve flight control at the bottom. The lower-level strategies simplify exploration.

The upper- and lower-level strategy cooperate with each other, and each plays its own role, which simplifies the exploration significantly and improves the training efficiency. Weight sharing is a very effective technique in training multi-agent strategies.

In the current multi-UAV air combat confrontation environment, the model parameters of all the UAVs are consistent, satisfying the homogeneity hypothesis. The local observations of all the agents are represented by Eq. (1), the global state is represented by Eq. (3), the upper action space is continuous space, and the upper action of agent a is represented as follows:

$$u_{upper}^a = \{ \underbrace{x, y, z}_{\text{Virtual position}}, \underbrace{\phi, \theta, \psi}_{\text{Virtual attitude}} \} \quad (13)$$

To sum up, take one agent of 3V3 scene as an example, we define the upper-level state as

$$\begin{aligned} o_{a1_self} &= \{x_{a1}, y_{a1}, z_{a1}, \phi_{a1}, \theta_{a1}, \psi_{a1}, h_{a1}\} \\ o_{a1_friends} &= \{o_{a2_self}, o_{a3_self}\} \\ o_{opponents} &= \{o_{b1_self}, o_{b2_self}, o_{b3_self}\} \\ o^{a1} &= \{o_{a1_self}, o_{friends}, o_{opponents}\} \end{aligned} \quad (14)$$

The output action of upper-level agent is

$$u_{upper}^{a1} = \{x_{vb1}, y_{vb1}, z_{vb1}, \phi_{vb1}, \theta_{vb1}, \psi_{vb1}\} \quad (15)$$

The upper-layer agent outputs a virtual opponent of the agent, and then lower-level state is defined as

$$o_{lower_a1} = u_{upper}^{a1} \quad (16)$$

The output action of lower-level agent is

$$u^{a1} = \{\delta_a^{a1}, \delta_r^{a1}, \delta_e^{a1}, \delta_t^{a1}\} \quad (17)$$

Communication between high and low level agents is shown in Fig. 3: the upper strategy output the upper action, which is passed to the lower level as the state. Then the lower agent output actions according to the pretrained strategy.

4.2. Value attention decomposition

The strategy gradient method, which has better convergence, is widely used in the fields of robotics and games. In single-agent reinforcement learning, the agent obtains the observation from the environment, generates the action according to its own strategy, and the environment executes the action to complete the state transfer and return the single-step reward to the agent. The simplest strategy gradient method REINFORCE has been proposed. The parameter θ_π of the parameterized strategy is updated by estimating the gradient of

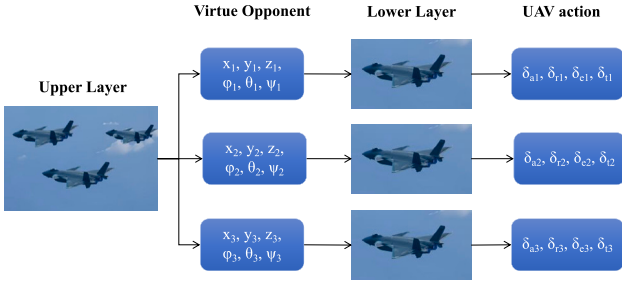


Fig. 3. Agent communication structure: upper strategy output the upper action, which is passed to the lower level as the state. Then the lower agent output actions.

the expected discount cumulative return $J = E_{\pi} [R_0]$. The gradient estimation formula of the algorithm is as follows:

$$\Delta_{\theta} U(\theta) \approx \hat{g} = \frac{1}{m} \sum_{i=1}^m \Delta_{\theta} \log P(\tau^{(i)}; \theta) R(\tau^{(i)}) \quad (18)$$

In the gradient estimation formula, $\Delta_{\theta} \log P(\tau; \theta)$ represents the direction in which the probability of the trajectory τ changes the most as the policy parameter θ changes, which is the gradient of the probability of the trajectory τ to the parameter θ . $R(\tau)$ represents the cumulative return of the corresponding trajectory, and the greater the cumulative return, the greater the probability of updating the corresponding trajectory. Therefore, the gradient calculated using Eq. (10) meets the expectation that the strategy will be updated in the direction of the trajectory that rewards more. However, the strategy gradient method leads to large variance; therefore, a baseline is introduced to reduce the variance of the gradient. The gradient in Eq. (10) becomes $\Delta_{\theta} U(\theta) \approx \hat{g} = \frac{1}{m} \sum_{i=1}^m \Delta_{\theta} \log P(\tau; \theta) (R(\tau) - b)$, where the constant b represents the baseline, b can also be written as $b(s_t)$, and the baseline is a value independent of the agent action u_t . In the actor-critic algorithm structure combined with the policy gradient and value function method, the actor is the parameterized strategy, critic is the approximator of the estimated value function, and the update gradient formula of actor strategy is as follows:

$$\Delta_{\theta} U(\theta) \approx \hat{g} = \frac{1}{m} \sum_{i=1}^m \Delta_{\theta} \log P(u_i | s_i) \underbrace{(Q(s_i, u_i) - V(s_i))}_{\text{advantage function}} \quad (19)$$

where $P(u_i | s_i)$ is the probability of taking action u_i at state s_i , $V(s_i)$ is the value function at state s_i and is often used as the baseline to reduce the variance, and $Q(s_i, u_i) - V(s_i)$ is regarded as the dominant function $A(s_i, u_i)$.

Unlike the single-agent environment, in a fully cooperative multi-agent environment, there is a global state s_t at time t , and each agent obtains a local observation o_t related to itself and makes a decision based on it. Thereafter, all agent actions are combined into joint actions acting on the environment, and the environment completes the state transition and returns a team reward r . This leads to the difference in actor-critic algorithm structure between a single- and cooperative multi-agent environment. Fig. 4 illustrates the actor-critic algorithm structure in a single-agent environment. The blue actor acts as the strategy of the agent and calculates the action u to be taken according to the global state s . The environment returns the corresponding reward r and the next state s' after completing the state transition. The green critic is used to train to fit the value function under the current strategy. The estimate of the value function is related to the state, as indicated by the arrow from the green dotted box in the figure.

There are usually three ways to estimate the value function without a model: based on the time difference of the single step temporal-difference (TD) learning algorithm, based on the complete path sequence of the Monte Carlo (MC) method, and the between TD (λ) algorithm. The different methods have different properties. The MC

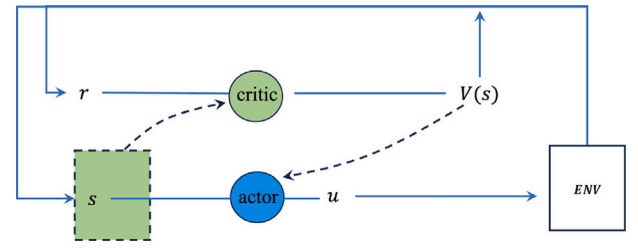


Fig. 4. Single-agent actor-critic structure: The actor selects actions based on the current policy, whereas the critic assesses the values of those actions and provides feedback to help the actor adjust its strategy.

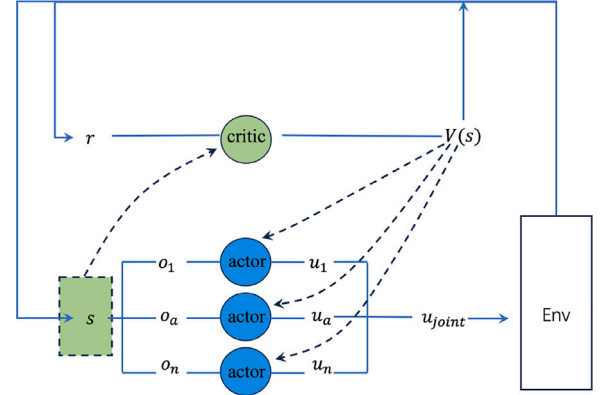


Fig. 5. Multi-agent actor-critic structure: The actors select actions, and the critic assesses the value of those actions and provides feedback to help each actor adjust its strategy.

method can calculate the return after obtaining the complete sequence of trajectories, without bias but with high variance, and does not take advantage of the Markov properties. The TD method makes full use of the Markov property and can use incomplete trajectory sequences to calculate returns, with bias but small variance. The blue actor is updated through policy gradients, which need to use the value function to compute the advantage function to reduce the variance, as indicated by the black dashed arrow on the right side in Fig. 4.

Fig. 5 depicts the actor-critic algorithm structure used in a cooperative multi-agent environment. The environment has a global state s , and each agent can obtain its own local observation o from the global state and calculate individual actions using the corresponding actor strategy. The environment executes the joint action u_{joint} and completes the state transfer; it returns a global reward and the next global status. The training of the value function depends only on the global state s and global reward r . Therefore, there is only one value function, but the strategy of each agent is different, and the training of all the actor strategies uses the same value function to calculate the advantage function. Thus, it cannot truly measure the contribution of each agent to the team return. The three black dotted arrows on the right side of the picture indicate.

In this section, we adopt an actor-critic structure of independent advantage function, as illustrated in Fig. 6, which is mainly divided into two parts. The upper part of the figure depicts the value function estimation, and the lower part illustrates strategy calculation, both in a cooperative multi-agent environment. In such an environment, the global state, global rewards, and joint actions are unique, whereas the multi-agent strategy is distributed because each agent is assigned a separate actor.

To measure the contribution of each agent action to the team return, an independent value function is set for each agent, and all the value functions are finally summarized into the global value function through

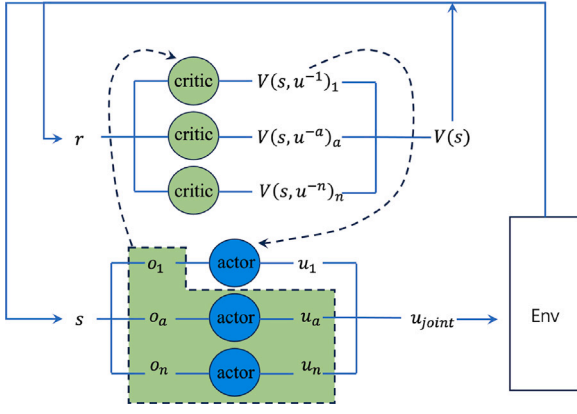


Fig. 6. Multi-agent independent advantage function: The actors select actions, and the critic assesses each actor through the independent advantage function; it then provides feedback to each actor and adjusts their strategies.

the network, as depicted in the upper part of the figure. The policy actor of each agent calculates the advantage function using the corresponding value function separately when updated, as indicated by the arrow at the back of the figure, thus avoiding the credit assignment problem caused by all agents using the same value function, as depicted in Fig. 5. Each agent value function is a function of all the agent observations (global state) and all other agent actions, as indicated by the arrow on the left. This actor-critic algorithm structure of the independent dominant function extends the single-agent actor-critic structure to the multi-agent environment. Therefore, we extend the single-agent strategy update formula (Eq. (20)), and the gradient update formula for agent a is as follows:

$$\Delta_{\theta} U^a(\theta) \approx \hat{g}_a = \frac{1}{m} \sum_{i=1}^m \Delta_{\theta} \log P(u_i^a | o_i^a) \quad (20)$$

$$* (Q(s_t, u_t^{-a}, u_t^a) - V(s_t, u_t^{-a}))$$

where a indicates the agent number, u_t^a indicates the action taken by agent a at time t , and u_t^{-a} indicates the action taken by all other agents excluding agent a at time t . For agent a , the global state s and all other agent actions u_t^{-a} can be uniformly regarded as the state of the environment, defined as the generalized state of agent a : $s_a = s, u^{-a}$. Therefore, the dominant function in the formula can be written as $Q(s_a^a, u_t^a) - V(s_a^a)$. Thus, it has a unified form with the actor policy update formula of a single agent.

In the implementation of the algorithm, the advantage function of each agent is calculated using the generalized advantage estimation. The formula for calculating the advantage function of agent a is as follows:

$$\hat{A}_t^{GAE(\gamma, \lambda), a} = \sum_{l=1}^{\infty} (\gamma \lambda)^l \delta_{t+l}^V \quad (21)$$

$$= \sum_{l=1}^{\infty} (\gamma \lambda)^l (r_t + \gamma V(s_{t+l+1}^a) - V(s_{t+l}^a))$$

To avoid the rapid update of each strategy, the clipping method in the PPO algorithm is used to calculate the loss. The strategy loss function of agent a is as follows:

$$L^a(\theta) = E \left[\min \left(w_{\theta, t}^a \hat{A}_t^{GAE(\gamma, \lambda), a}, \text{clip} \left(w_{\theta, t}^a, 1 - \epsilon, 1 + \epsilon \right) \hat{A}_t^{GAE(\gamma, \lambda), a} \right) \right] \quad (22)$$

The clip function can clip the data to the specified range. $1 - \epsilon$ and $1 + \epsilon$ represent the upper and lower limits of the clipping range, respectively. $w_{\theta, t}^a$ are the weight factors of the old and new policies of agent a at time

t , and they can limit the degree to which the updated policy deviates from the old policy, which is defined as follows:

$$w_{\theta, t}^a = \frac{\pi_{\theta}^a(u_t^a | o_t^a)}{\pi_{\theta_{old}}^a(u_t^a | o_t^a)} \quad (23)$$

where π_{θ}^a represents the policy of agent a , and θ is the parameter that needs to be updated in the policy.

The degree of contribution of the actions of each agent to the team may be positive or negative. The global state-value function of the team can be obtained from the local value functions of all the agents through comprehensive operations. As depicted in Fig. 7, we design an attention-based value function network. The left part of the diagram is the V (value) of the attention mechanism. The generalized state specific to each agent $s_i^a, a \in A \equiv 1, \dots, n$ is input; following the multilayer perceptron, gated loop unit, and output layer multilayer perceptron, the output value function $V(s_i^a)$ corresponding to each agent is composed of the value function vector. This part corresponds to a local critic network; each agent has a local critic network.

The middle part serves as the key (K) of attention; it inputs the generalized state unique to each agent and outputs the key in the hidden space through network transformation. The part on the right is the query (Q) of the attention mechanism, and the global state of the input is transformed by the network to obtain the query in the hidden space. Q and all K are inner products; therefore, the attention weight is obtained through the softmax network layer. The global value function can be calculated using the inner product of the attention weight and value function vector. To measure the positive and negative contribution of each agent to the global return, a parallel two-layer network is designed for the Q and K parts of the attention mechanism. After two layers of softmax, two sets of weights are output. In the figure, the weights marked “+” are positive, whereas those marked “−” are negative. The two sets of weights are internally integrated with the local value function network to obtain the global positive and negative contribution values, respectively, and the global value function V_{total} is summed between them:

$$w^+ = \text{softmax}^+(Query_1(s_t) \top Key_1(\{s_1^1, \dots, s_t^n\}))$$

$$w^- = \text{softmax}^-(Query_2(s_t) \top Key_2(\{s_1^1, \dots, s_t^n\})) \quad (24)$$

$$V^{total} = Value(\{s_1^1, \dots, s_t^n\}) \top w^+ + Value(\{s_1^1, \dots, s_t^n\}) \top w^-$$

where Query1(·) and Query2(·) represent two parameterized query networks in the attention mechanism, Key1(·) and Key2(·) represent two parameterized key networks in the attention mechanism, and w^+ and w^- represent two sets of attention weights. Value(·) represents the Value network in the attention mechanism.

Algorithm 1 Framework of upper-level strategy training

Input: Number of agents N , training epoch M , initial upper-level strategy $\pi_{a1}, \dots, \pi_{an}$, pretrained lower-level strategy π_r , game time limit T

Output: The reinforced upper-level strategy $\pi_{a1}^*, \dots, \pi_{an}^*$

- 1: **for** $i = 1 \rightarrow M$ **do**
- 2: Initial global state $s(0)$
- 3: **for** $t = 0 \rightarrow T$ **do**
- 4: **for** $j = 1, 2, \dots, N$ **do**
- 5: $o^{aj} = (o_{ai_self}, o_{ai_friends}, o_{opponents})$
- 6: $u_{upper}^{aj} = \pi_{a1}^{aj}(o^{aj})$
- 7: $o_{lower_ai} = u_{upper}^{aj}$
- 8: $u^{aj} = \pi_r(o_{lower_ai})$
- 9: $u^{red}(t) = (u^{a1}, \dots, u^{an})$
- 10: $r(t), s(t+1) = G(u^{red}(t), s(t))$
- 11: Update $\pi_{a1}, \dots, \pi_{an}$ with the reinforcement learning by minimizing the loss function $L(w)$
- 12: **return** C_1, C_2

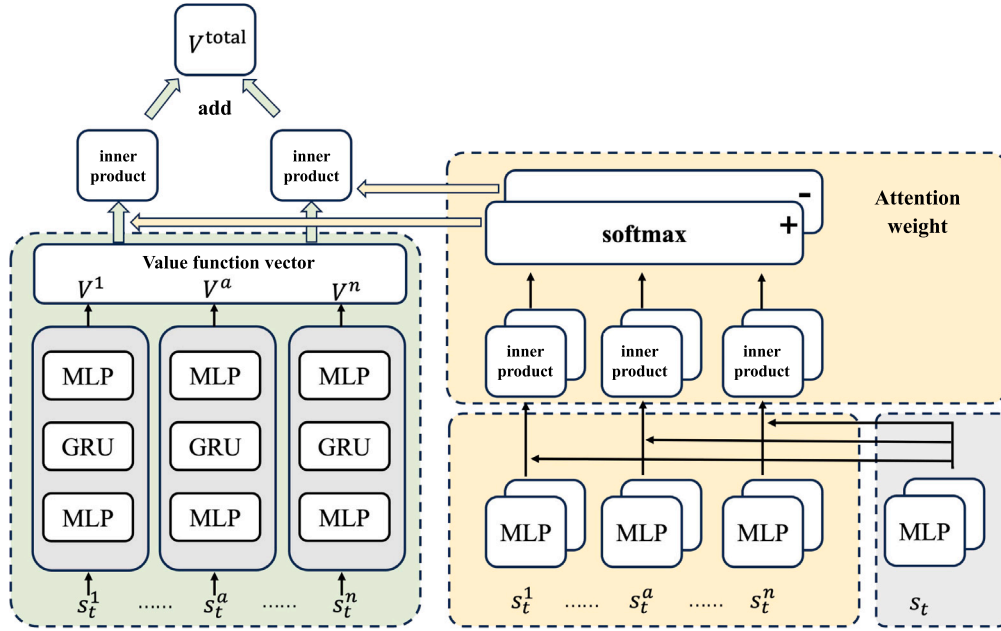


Fig. 7. Value function network: The left part of the diagram serves as the value (V) component of the attention mechanism. It takes the generalized state specific to each agent as the input. Through a multilayer perceptron (MLP), the middle part functions as the key (K) for attention, transforming the specific generalized state of the agent into the key in the hidden space. On the right side, the query (Q) component of the attention mechanism takes the input of the global state and transforms it through a network to obtain the query in the hidden space. The inner product of Q with all the K values, followed by a softmax layer, yields the attention weights. These weights are used with the value function vector to calculate the global value function. To assess the contribution of each agent to the global reward, parallel two-layer networks are designed for the Q and K parts of the attention mechanism. After passing through two softmax layers, two sets of weights are obtained; the weights marked “+” are positive and those marked “-” are negative. These weights are used to compute the positive and negative contributions to the global value function, which are summed to derive the total global value function V total.

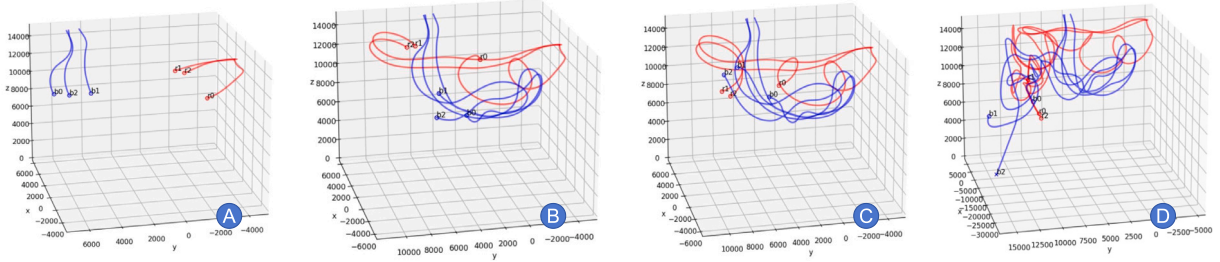


Fig. 8. Result of a confrontation in Pyaircombat: The red and blue UAVs represent the hierarchy- and rule-based policies, respectively. (A) The red and blue UAVs start from the initial position; (B) the red UAVs are divided into two teams to fight with the blue UAVs; (C) the red and blue UAVs begin to wrestle; (D) the red UAVs annihilate the blue UAV b2, the battle time reaches the upper limit, and the confrontation ends. In the course of the confrontation, the upper level strategy-controlled UAV jointly selected the collectivized UAV b2 and defeated the opponent through cooperation.

We elaborate on the framework of upper-level strategy training through Algorithm 1.

The loss function of the total value function is as follows:

$$L(\omega) = \frac{1}{n} \sum_{i=1}^n \max[(r_t - V_{\omega}(s_t))^2, (r_t - \text{clip}(V_{\omega}(s_t) - \hat{V}_t, 1 - \epsilon, 1 + \epsilon))^2] \quad (25)$$

5. Experiments

5.1. Experimental environment

Pyaircombat is a high-fidelity air combat simulation framework developed for BVR intelligent air combat, based on Unity and JSBSim. This platform encompasses UCAV control, fire control, and reconnaissance systems. Pyaircombat offers a high-fidelity environment with a significant accelerative ratio, making it the preferred choice for our project.

5.2. Parameter setting

To verify the multi-agent method, we designed a simulation environment for many-on-many confrontation of fixed-wing UAVs. The fixed wing UAVs in the environment can be divided into red and blue camps, and the number of fixed wing UAVs on both sides can be configured. The control instructions of the fixed wing UAVs are ailerons, rudder, elevator and throttle. These control quantities are specified by the strategy trained by the reinforcement learning multi-agent algorithm to control the UAV flight confrontation. The damage calculation formula and reward function for the enemy in the environment are described in detail in the preceding section. This chapter provides four different environments for the verification of the algorithm, namely, 3v3 adversarial environment with three red and blue fixed wing drones, 4v4 adversarial environment with four red and blue UAVs, 5v5 adversarial environment with five red and blue UAVs, and 6v6 adversarial environment with six red and blue UAVs. Among them, the red fixed-wing UAV is controlled by a multi-agent algorithm strategy, whereas the blue UAV is controlled by a built-in rule-based algorithm. The environment needs to be randomly initialized each time

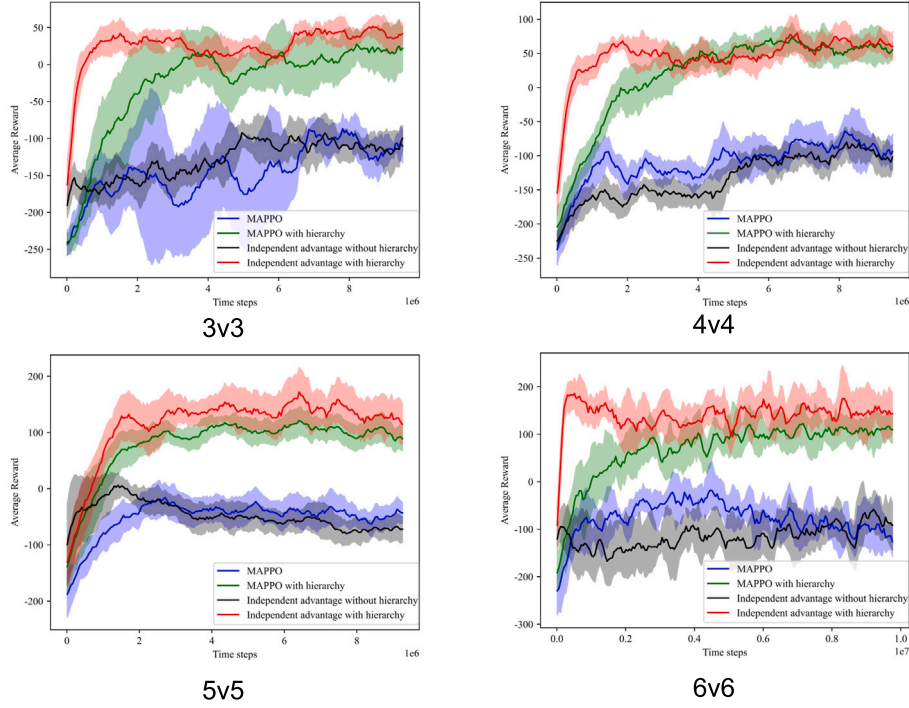


Fig. 9. 3v3, 4v4, 5v5, and 6v6 training curves obtained under identical training environments, except for the number of UAVs. In general, the red and green curves are better than the blue and black curves.

Table 1

Parameter values setting.

Name	Symbol	Value
Activation function	f	Relu
Number of layers	r_layers_N	2
Length of time block	relent	10
Learning rate	lr	5e-4
RMSprop optimizer epsilon	RMSprop ϵ	1e-5
Clip cut epsilon	clip ϵ	0.2
Loss coefficient of entropy	entropy_coef	0.01
Loss coefficient of value	value_coef	1
Discount factor	γ	0.99
λ of the advantage estimate	gae λ	0.95

it is started. All UAVs are initialized within a radius of 10 km, the red and blue UAVs are divided into two camps in terms of position, and the attitude of all UAVs is randomly set within a certain range to maintain an equal altitude advantage for both sides; all the drones are initialized to the same altitude.

The baseline algorithm widely applied to the current strategy of MAPPO is an extension of the single-agent proximal policy optimization (PPO). In the field of multi-agent systems, the MAPPO algorithm, with its normalized value function, generalized estimation, gradient, and skills has good practicability. To verify the effectiveness of the proposed method, we performed ablation experiments in four different scale environments. The parameters used in the algorithm training process are presented in Table 1.

5.3. Experiment results

To verify the effectiveness of the algorithm, the algorithm was applied in four environments of different sizes (3v3, 4v4, 5v5, and 6v6) for training. The training curve is presented in Fig. 9, where the horizontal axis represents the number of samples sampled during the training process and the vertical axis represents the average reward that changes with the number of samples. The larger the reward of

the vertical axis, the better the training effect of the algorithm and the larger the reward of the trained strategy in the confrontation. The smaller the number of samples, the larger the reward, the faster the algorithm convergence, and the easier the algorithm is to train. Fig. 8 depicts result of a 3v3 confrontation in Pyaircombat.

The red and green curves have higher reward values, whereas the blue and black curves have negative reward values, indicating that the hierarchical method significantly aids the training of multi-UAV air combat confrontation. Further, it alleviates the problem that the state action space is large and difficult to explore effectively in the confrontation of multi-UAV air combat. In addition, the red curve converges faster and has a higher reward value than the green curve, indicating that the use of an independent advantage function attention value network in a cooperative environment is conducive to training and can better train multiple agents to obtain higher rewards, thus alleviating the credit allocation problem in multi-agent cooperation scenarios to a certain extent. The blue curve indicated a large variance in the training process, which may be due to the small number of agents in the 3v3 environment; these could not stably explore a good cooperation strategy.

Fig. 9 depicts the training curves obtained in the 4v4, 5v5 and 6v6 environments. In general, the red and green curves are better than the blue and black curves, indicating the effectiveness of stratification in multi-UAV air combat confrontation. The convergence speed of the red curve is faster than that of the green curve, and the reward value is greater than that of the green curve, indicating that the independent advantage function method using the attention value network can alleviate the credit allocation problem to a certain extent.

After training with the same number of samples, the trained strategy is used to fight against the blue UAV. Each method in each environment needs to repeatedly fight 100 rounds, and the rewards obtained in each round are recorded and counted, depicted in the box diagram presented in Fig. 10. The figure indicates the reward distribution of strategies trained using various methods against the test in the 3v3 environment. After 100 repeated tests, the nonhierarchical method has

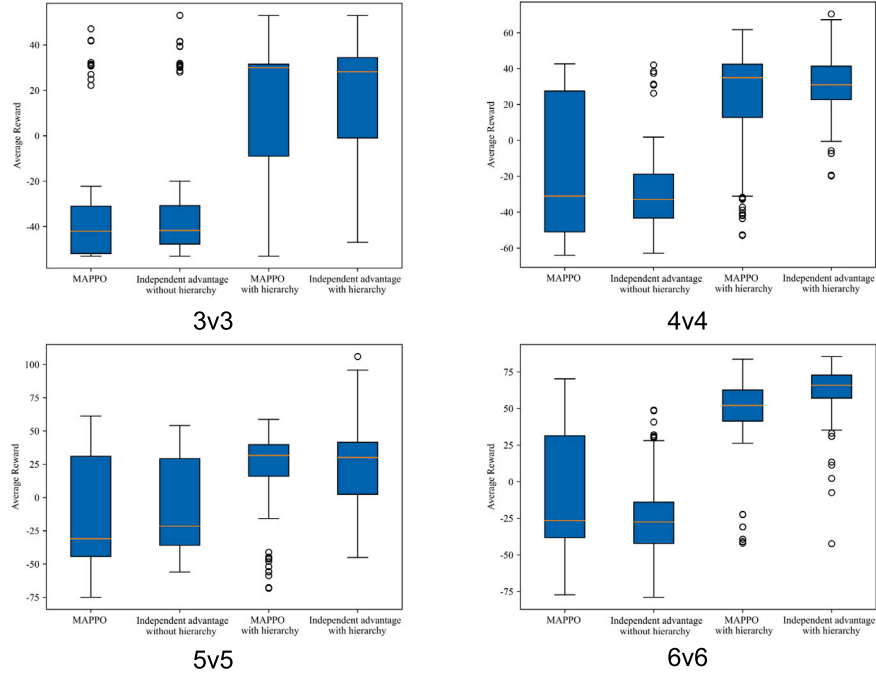


Fig. 10. Reward values for 100 rounds of 3v3, 4v4, 5v5, and 6v6 tests: Rewards obtained under identical training environments, except for the number of UAVs. The hierarchy structure and an independent advantage function approach generates higher reward values.

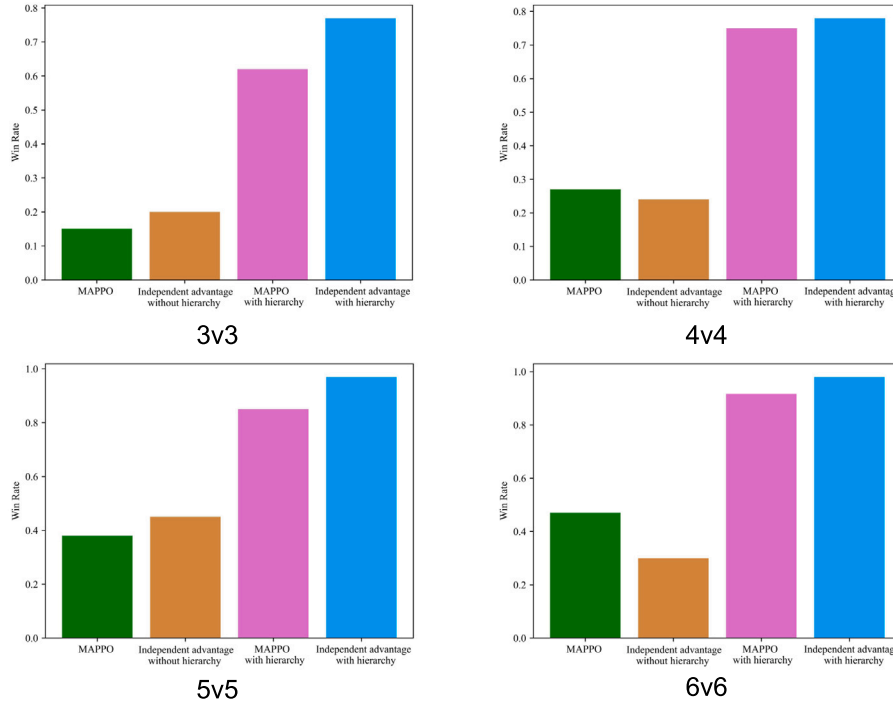


Fig. 11. Winning rates for 100 rounds of 3v3, 4v4, 5v5, and 6v6 tests: Winning rates under identical training environments, except for the number of UAVs. The success rate of the hierarchy structure is significantly higher than that of the method without hierarchy.

a large proportion of negative rewards, whereas a small number of positive rewards are outliers. A combination of the layered and independent advantage function approach generates higher reward values. The reward value data obtained using various methods for 100 adversarial tests repeated in the 4v4, 5v5, and 6v6 environments are presented in the figure. In general, the layered approach in many-on-many UAV adversarial tests can obtain higher rewards when compared with

the nonlayered approach. A combination of layered and independent advantage functions earned the highest rewards in adversarial testing.

In addition to the comparison of training curves, the win rate was also compared. The trained strategies were used in four environments of different scales to compete against the built-in rule strategies of the environment. In each environment, each algorithm repeatedly fought 100 games. Fig. 11 indicates that, independent of the environment, the winning rate of the layered approach is significantly higher than that

of the nonlayered approach. The layered approach combined with an independent advantage function can further improve the winning rate of the agent in the cooperation scenario.

6. Conclusion

To address the challenge of deriving the cooperative policy of friendly aircraft and the training difficulties caused by a large state action space in the multi-UAV air combat confrontation environment, a novel reinforcement learning method that combines virtual opponents and value attention decomposition is proposed. The upper layer uses a multi-agent reinforcement learning algorithm for centralized training and distributed execution to analyze the situation. The upper-level strategy outputs the virtual opponent state for the lower-level strategy based on the environment; therefore, the multi-UAV air combat confrontation problem is decomposed into a one-to-one confrontation problem. Finally, the lower-level strategy outputs the flight control command of the UAV to complete the one-to-one confrontation with the virtual opponent.

Furthermore, to solve the credit assignment problem in multi-agent reinforcement learning in cooperative scenarios, an independent advantage function method of the attention value network is adopted to learn the independent advantage function for each distributed policy network, and the attention method is used to learn the global state value function. Finally, the algorithm is trained and tested in the UAV combat simulation environment, and the effectiveness of the proposed method is verified by comparison.

CRedit authorship contribution statement

Xiaojie Xu: Conceptualization, Methodology, Software, Writing – original draft, Writing – review & editing. **Yunfan Wang:** Methodology, Funding acquisition, Writing – review & editing, Supervision. **Xian Guo:** Methodology, Writing – review & editing. **Kuihua Huang:** Supervision, Writing – review & editing. **Xuebo Zhang:** Supervision, Writing – review & editing.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Xian Guo reports financial support was provided by National Natural Science Foundation of China. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported by the National Natural Science Foundation of China under Grant 62073176.

Data availability

Data will be made available on request.

References

- Adrian, P. P., Jaime, S. I., Daria, M., Henry, D., Jason, T., Kevin, A., et al. (2023). Hierarchical reinforcement learning for air combat at DARPA's AlphaDogfight trials. *IEEE Transactions on Artificial Intelligence*, 4(6), 1371–1385.
- Barto, A. G., & Mahadevan, S. (2003). Recent advances in hierarchical reinforcement learning. *Discrete Event Dynamic Systems*, 13, 341–379.

- Bei, P., Tabish, R., Christian, W., Pierre, K., Philip, T., Wendelin, B., et al. (2021). Facmac: Factored multi-agent centralised policy gradients. *Advances in Neural Information Processing Systems*, 12208–12221.
- Berner, C., Greg, B., Brooke, C., Vicki, C., Przemyslaw, D., Christy, D., et al. (2019). Dota 2 with large scale deep reinforcement learning. arxiv preprint arxiv:1912.06680.
- Botvinick, M. M. (2012). Hierarchical reinforcement learning and decision making. *Current Opinion in Neurobiology*, 22(6), 956–962.
- Burgin, G. H., Fogel, L. J., & Phelps, J. P. (1975). *An adaptive maneuvering logic computer program for the simulation of one-on-one air-to-air combat. volume 1: General description: Technical report*, NASA.
- Carrio, A., Sampedro, C., Rodriguez-Ramos, A., Campoy, P., et al. (2017). A review of deep learning methods and applications for unmanned aerial vehicles. *Journal of Sensors*, 2017.
- Chao, Y., Akash, V., Eugene, V., Jiaxuan, G., Yu, W., Alexandre, B., et al. (2022). The surprising effectiveness of ppo in cooperative multi-agent games. *Advances in Neural Information Processing Systems*, 24611–24624.
- Chenxu, Q., Xuebo, Z., Lun, L., Minghui, Z., & Yongchun, F. (2024). H3E: Learning air combat with a three-level hierarchical framework embedding expert knowledge. *Expert Systems with Applications*, 245(2024), Article 123084.
- Ernest, N., Carroll, D., Schumacher, C., Clark, M., Cohen, K., & Lee, G. (2016). Genetic fuzzy based artificial intelligence for unmanned combat aerial vehicle control in simulated air combat missions. *Journal of Defense Management*, 6(1), 2167–2374.
- Floyd, M. W., Karneeb, J., Moore, P., & Aha, D. W. (2017). A goal reasoning agent for controlling UAVs in beyond-visual-range air combat. Vol. 17, In *IJCAI* (pp. 4714–4721).
- Gong, Z., Xu, Y., & Luo, D. (2023). UAV cooperative air combat maneuvering confrontation based on multi-agent reinforcement learning. *Unmanned Systems*, 11(03), 273–286. <http://dx.doi.org/10.1142/S2301385023410029>.
- Haiyin, P., Yue, H., Hechang, C., Xuanqi, P., Songyuan, F., Yang, S., et al. (2023). Complex relationship graph abstraction for autonomous air combat collaboration: A learning and expert knowledge hybrid approach. *Expert Systems with Applications*, 215(2023), Article 119285.
- Haiyin, P., Yue, H., Shaoming, H., Chao, Y., Fan, S., & Yaqing, H. (2023). Spatiotemporal relationship Cognitive Learning for multirobot air combat. *IEEE Transactions on Cognitive and Developmental Systems*, 15(4), 2254–2268.
- Harmon, M. E., Baird, L. C., III, & Klop, A. H. (1995). Reinforcement learning applied to a differential game. *Adaptive Behavior*, 4(1), 3–28.
- Isaacs, R. (1951). *Games of pursuit*. Santa Monica, CA: RAND Corporation.
- Jiandong, Z., Qiming, Y., Guoqing, S., Yi, L., & Yong, W. (2021). UAV cooperative air combat maneuver decision based on multi-agent reinforcement learning. *Journal of Systems Engineering and Electronics*, 32(6), 1421–1438.
- Julien, P., Bart, V., Daniel, H., Eugene, T., Florian, S., Vincent, B., et al. (2022). Mastering the game of stratego with model-free multiagent reinforcement learning. *Science*, 378(6623), 990–996.
- Junjie, L., Sotetsu, K., Qiwei, Y., Guoqing, L., Chao, W., Ruihan, Y., et al. (2020). Suphx: Mastering mahjong with Deep Reinforcement learning. arxiv preprint arxiv:2003.13590.
- Kyunghwan, S., Daewoo, K., Wan, K., David, H., & Yung, L. (2019). QTRAN: Learning to factorize with transformation for cooperative multi-agent reinforcement learning. In *International conference on machine learning* (pp. 5887–5896).
- Liu, C., Sun, S., Tao, C., Shou, Y., & Xu, B. (2021). Sliding mode control of multi-agent system with application to UAV air combat. *Computers & Electrical Engineering*, 96, Article 107491.
- Oriol, V., Igor, B., Wojciech, C., Michael, M., Andrew, D., Junyoung, C., et al. (2019). Grandmaster level in StarCraft II using multi-agent reinforcement learning. *Nature*, 575(7782), 350–354.
- Peter, S., Guy, L., Audrunas, G., Wojciech, C., Vinicius, Z., Max, J., et al. (2017). Value-decomposition networks for cooperative multi-agent learning. arxiv preprint arxiv:1706.05296.
- Pham, H. X., La, H. M., Feil-Seifer, D., & Nguyen, L. V. (2018). Autonomous uav navigation using reinforcement learning. arXiv preprint arXiv:1801.05086.
- Pope, A. P., Ide, J. S., Mićović, D., Diaz, H., Twedt, J. C., Alcedo, K., et al. (2022). Hierarchical reinforcement learning for air combat at DARPA's AlphaDogfight trials. *IEEE Transactions on Artificial Intelligence*.
- Silver, D., Schrittwieser, J., Simonyan, K., Antonoglou, I., Huang, A., Guez, A., et al. (2017). Mastering the game of go without human knowledge. *Nature*, 550(7676), 354–359.
- Tabish, R., Mikayel, S., Christian, W., Gregory, F., Jakob, F., & Shimon, W. (2020). Monotonic value function factorisation for deep multi-agent reinforcement learning. *Journal of Machine Learning Research*, 21(178), 1–51.
- Virtanen, K., Karelaiti, J., & Raivio, T. (2006). Modeling air combat by a moving horizon influence diagram game. *Journal of Guidance, Control, and Dynamics*, 29(5), 1080–1091.
- Yang, J., Yang, X., & Yu, T. (2024). Multi-unmanned aerial vehicle confrontation in intelligent air combat: A multi-agent deep reinforcement learning approach. *Drones*, 8(8), 382.
- Zheng, Y., Xin, B., He, B., & Ding, Y. (2024). Mean policy-based proximal policy optimization for maneuvering decision in multi-UAV air combat. *Neural Computing and Applications*, 36(31), 19667–19690.